

Algorithm:

Step 1: Initialization

- a. Input n (size of board)
- b. Create:
 - board[n][n] filled with '.'
 - col[n] → tracks used columns
 - diag1[2n-1] → tracks main diagonals (row - col + n - 1)
 - diag2[2n-1] → tracks anti-diagonals (row + col)

Step 2: Start Recursion

- a. Call: solve(0)

Step 3: Recursive Function: solve(row)

- a. Base Case:
 - if row == n:
 - count++
 - printBoard()
 - return
 - All queens placed successfully → valid solution
- b. Try placing queen in each column:
 - for c = 0 to n-1:

Step 4.: Safety Check (O(1))

Check if placing queen is valid:

if col[c] == false AND diag1[row - c + n - 1] == false AND diag2[row + c] == false

Step 5: Place Queen

- a. board[row][c] = 'Q'
- b. col[c] = true
- c. diag1[row - c + n - 1] = true
- d. diag2[row + c] = true

Step 6: Recurse for Next Row `solve(row + 1)`

Step 7: Backtrack

a. Undo placement:

`board[row][c] = '.'`

`col[c] = false`

`diag1[row - c + n - 1] = false`

`diag2[row + c] = false`

Step 8: Continue Trying All Possibilities